

Design and Analysis of Algorithms: CI43

Unit-V

NP and Computational Intractability

NP (Nondeterministic Polynomial Time)

Definition:

NP is the class of decision problems for which a solution, if given, can be verified in polynomial time by a deterministic Turing machine.

Example:

Given a Sudoku puzzle solution, checking if it's valid can be done quickly (in polynomial time), so Sudoku is in NP.

✓ Computational Intractability

Definition:

A problem is computationally intractable if there is no known algorithm that can solve all instances of the problem in polynomial time (i.e., efficiently) as the input size grows.

Context:

Many problems in NP are believed to be intractable because we don't know polynomial-time algorithms to solve them (unless $P = NP$, which is still unproven).

Relationship Between NP and Intractability

- All NP-complete and NP-hard problems are **considered intractable** because they **cannot (currently) be solved in polynomial time**.
- Intractability highlights the **limits of algorithmic efficiency** and motivates the study of **approximation algorithms, heuristics, and special cases**.

Explain P, NP and NP-Complete problems.

Basic Complexity Classes

Class	Definition
	Set of decision problems solvable in polynomial time.
NP	Set of decision problems for which a solution can be verified in polynomial time.
NP-Hard	Problems that are at least as hard as the hardest problems in NP. (Not necessarily in NP).
NP-Complete	Problems that are in NP and are NP-Hard. Solving any one in polynomial time solves all NP problems.

Explain P, NP and NP-Complete problems.

P Class

- The P in the P class stands for **Polynomial Time**. It is the collection of decision problems(problems with a “yes” or “no” answer) that can be solved by a deterministic machine (our computers) in polynomial time.
- **Features:**
- The solution to **P problems** is easy to find.
- **P** is often a class of computational problems that are solvable and tractable. Tractable means that the problems can be solved in theory as well as in practice. But the problems that can be solved in theory but not in practice are known as intractable.
- Most of the coding problems that we solve fall in this category like the below.
 - ☐ Calculating the greatest common divisor.
 - ☐ Finding a maximum matching.
 - ☐ Merge Sort

NP Class

What is an Approximation Algorithm?

An approximation algorithm is a fast (polynomial time) algorithm used to solve NP-Hard problems.

Since we can't find the exact (optimal) solution quickly, approximation algorithms give a "good enough" solution that is close to the best.

Approximation Ratio

This tells us how close the algorithm's answer is to the best answer.

- **For minimization problems:**

$$\text{Approximation Ratio} = \frac{\text{Algorithm's Answer}}{\text{Optimal Answer}} \leq \rho$$

- **For maximization problems:**

$$\frac{\text{Optimal Answer}}{\text{Algorithm's Answer}} \leq \rho$$

Where $\rho \geq 1$, and smaller ρ means better approximation.

Examples

1. **Vertex Cover**

- Pick any edge.
- Add both its endpoints to the cover.
- Repeat until all edges are covered.
- Approximation ratio = 2 (solution is at most $2\times$ the best one).

2. **Traveling Salesman Problem (TSP)** – with triangle inequality

- Build a Minimum Spanning Tree (MST).
- Do a DFS walk and make a tour.
- Approximation ratio = 2

3. **Knapsack Problem**

Circuit Satisfiability (SAT): The First NP-Complete Problem

✓ What is it?

- You are given a Boolean circuit made of AND, OR, and NOT gates.
- The question: Is there some input assignment (True/False) that makes the output True?

🔍 Why is it important?

- Circuit SAT was the first problem proved to be NP-Complete (by Stephen Cook in 1971).
- Every problem in NP can be converted (reduced) to Circuit SAT in polynomial time.

📖 2. General Strategy for Proving New Problems are NP-Complete

To prove a new problem P is NP-Complete, follow these 3 steps:

✓ Step 1: Show that P is in NP

- Prove that if someone gives you a solution, you can verify it in polynomial time.

✓ Step 2: Pick a known NP-Complete problem (like SAT, 3-SAT, or Vertex Cover).

✓ Step 3: Reduce the known NP-Complete problem to your problem P in polynomial time.

- This means: if you can solve P, you can solve the old problem too.
- It shows P is at least as hard as that NP-Complete problem.

🔄 3. Sequencing Problems

Sequencing problems are about arranging items (like jobs, cities, tasks) in a specific order, and many of them are NP-Complete.

What is TSP?

Imagine you're a salesman, and you have to visit a few cities. You want to:

Visit each city **once**.

Minimize the total **distance** you travel.

Return to your starting city at the end.

This is the **Traveling Salesman Problem** (TSP).

Graph Representation:

In a **graph**, you have:

Cities as dots (nodes).

Distances between cities as lines (edges), and the lines have a **weight** that represents the distance between cities.

Example:

Cities and Distances:

- $A-B = 10$
 - $A-C = 15$
 - $A-D = 20$
 - $B-C = 35$
 - $B-D = 25$
 - $C-D = 30$
-

✓ Goal:

Visit all 4 cities once, return to the starting city (A), and keep the total distance as small as possible.

Try all possible routes starting from A:

1. $A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

$\rightarrow 10 + 35 + 30 + 20 = 95$

2. $A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$

$\rightarrow 10 + 25 + 30 + 15 = \checkmark 80$ (Best)

3. $A \rightarrow C \rightarrow B \rightarrow D \rightarrow A$

$\rightarrow 15 + 35 + 25 + 20 = 95$

4. $A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$

$\rightarrow 15 + 30 + 25 + 10 = \checkmark 80$ (Best)

5. $A \rightarrow D \rightarrow B \rightarrow C \rightarrow A$

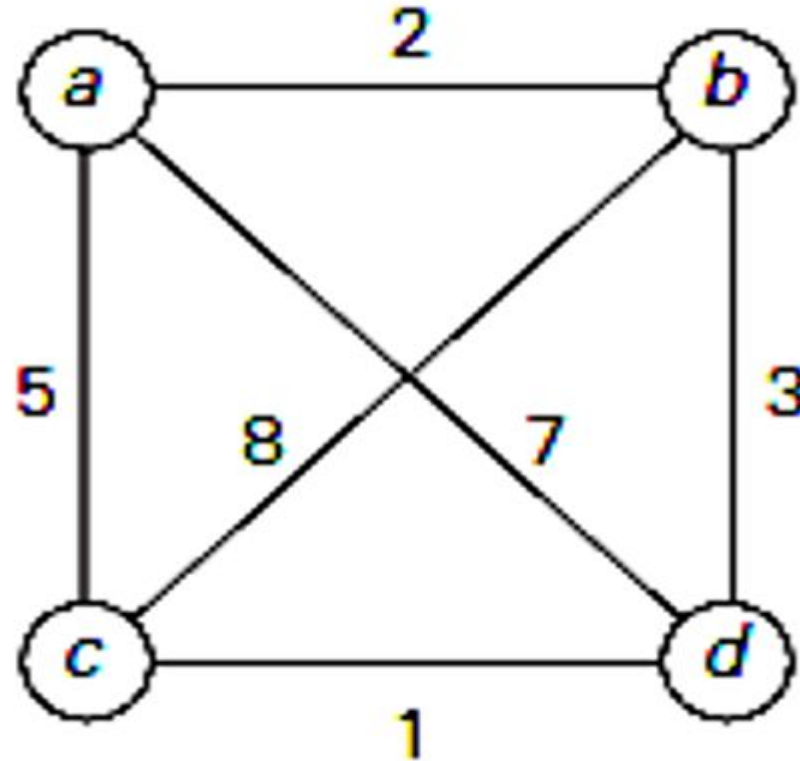
$\rightarrow 20 + 25 + 35 + 15 = 95$

6. $A \rightarrow D \rightarrow C \rightarrow B \rightarrow A$

$\rightarrow 20 + 30 + 35 + 10 = 95$

- **Final Answer (Shortest Distance):**
 - **80 miles**
 -  **Best Routes:**
 - **$A \rightarrow B \rightarrow D \rightarrow C \rightarrow A$**
 - **$A \rightarrow C \rightarrow D \rightarrow B \rightarrow A$**
 - Both give the same shortest total.

Apply the branch-and-bound algorithm to solve the traveling salesman problem for the graph given in Fig. considering 'a' source vertex:



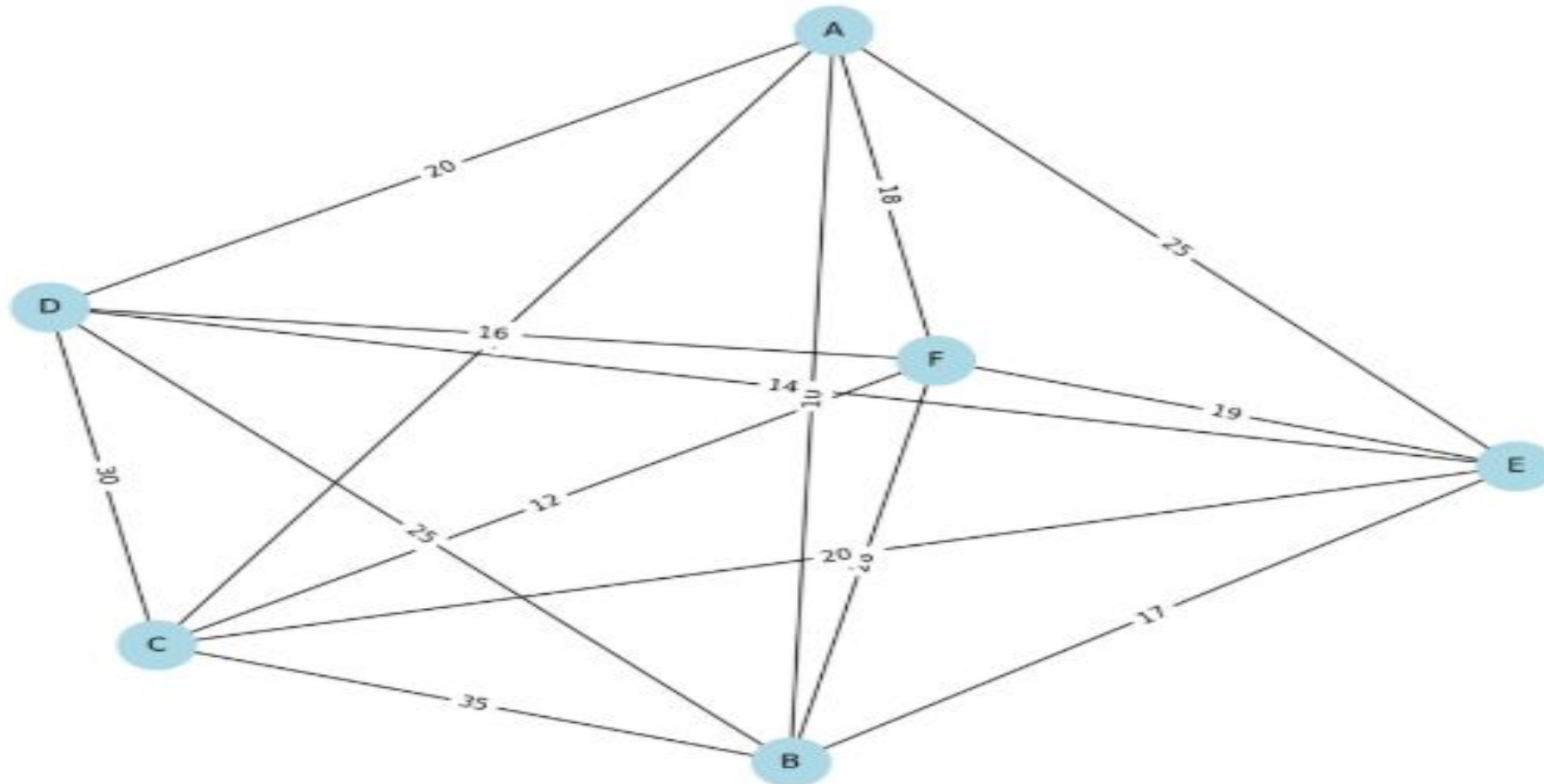
- $a \rightarrow b = 2$
- $a \rightarrow c = 5$
- $c \rightarrow d = 1$
- $b \rightarrow d = 3$
- $a \rightarrow d = 7$
- $c \rightarrow b = 8$
- Now, let's calculate the possible paths:
- **Possible Paths:**
- $a \rightarrow b \rightarrow c \rightarrow d \rightarrow a$
Distance: $2+8+1+7=18$

- $a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$
Distance: $2+3+1+5=11$
- $a \rightarrow c \rightarrow b \rightarrow d \rightarrow a$
Distance: $5+8+3+7=23$
- $a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$
Distance: $5+1+3+2=11$
- $a \rightarrow d \rightarrow b \rightarrow c \rightarrow a$
Distance: $7+3+8+5=23$
- $a \rightarrow d \rightarrow c \rightarrow b \rightarrow a$
Distance: $7+1+8+2=18$.
- Therefore Shortest Path is $a \rightarrow b \rightarrow d \rightarrow c \rightarrow a$ or $a \rightarrow c \rightarrow d \rightarrow b \rightarrow a$

Distance is 11

TSP Problem

TSP Graph with 6 Cities



- $A-D=20$
- $A-C=16$
- $A-F=18$
- $A-E=25$
- $D-F=16$
- $D-E=14$
- $C-B=35$
- $C-F=12$
- $B-E=17$
- $D-B=25$

- $E-F=10$

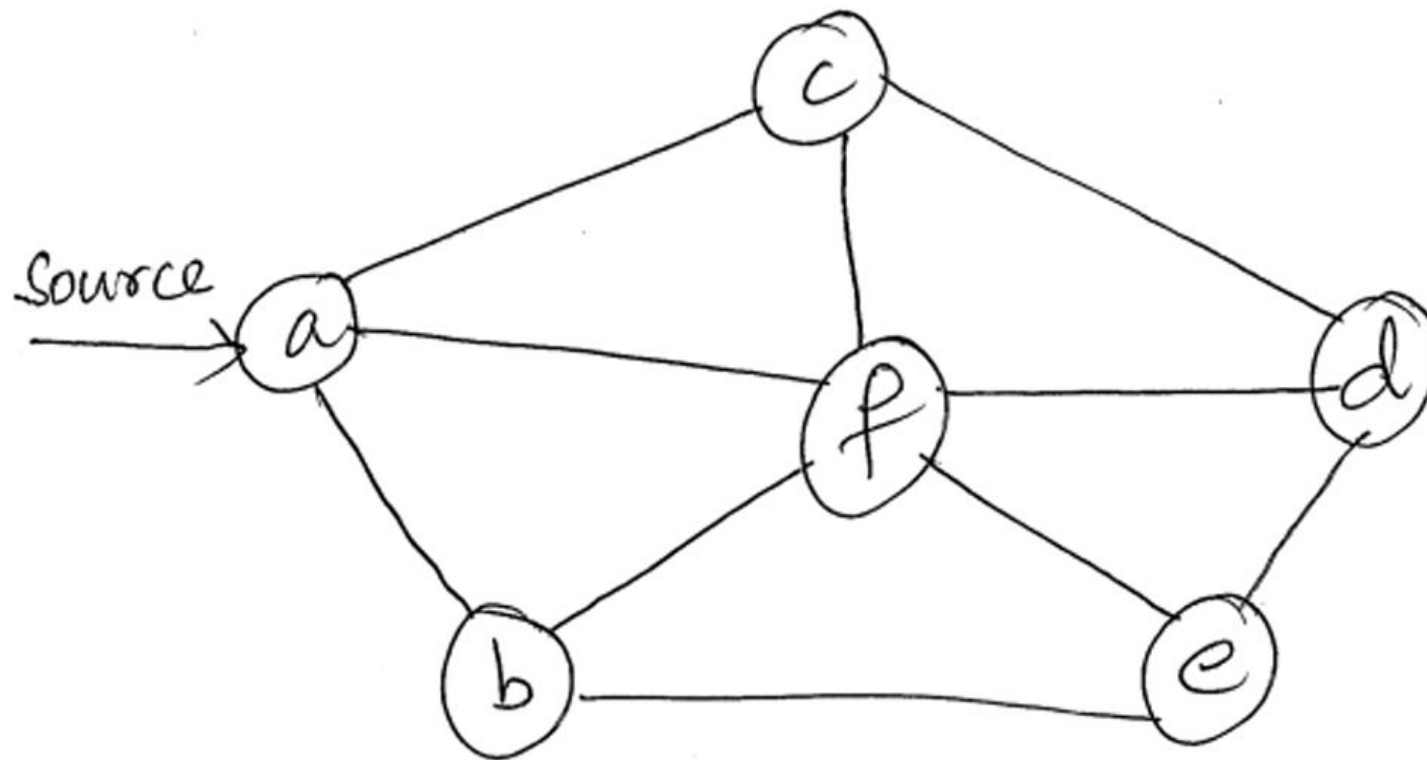
- Possible routes:
- $A \rightarrow C \rightarrow F \rightarrow D \rightarrow B \rightarrow E \rightarrow A$ $16 + 12 + 16 + 25 + 17 + 25 = 111$
- $A \rightarrow C \rightarrow E \rightarrow F \rightarrow B \rightarrow D \rightarrow A$ $16 + 20 + 19 + 20 + 25 + 20 = 120$
- $A \rightarrow F \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow A$ $18 + 12 + 35 + 25 + 14 + 25 = 129$

- **Minimum Distance Routes (109 units)**
- **A-C-F-D-B-E-A**
- **$16+12+14+25+17+25=109$**
- **A-C-F-B-E-D-A**
- **$16+12+20+17+14+20=99$**
- **Minimum distance=99**

A Hamiltonian cycle (or Hamiltonian circuit) in a graph is a simple closed loop that:

- Starts and ends at the same vertex.
- Visits every other vertex exactly once (no repeats except returning to the start).

Describe Hamiltonian cycle. Find a Hamiltonian cycle in the graph given in FIG



A valid Hamiltonian cycle starting and ending at the source a is:

$$| a \rightarrow b \rightarrow e \rightarrow d \rightarrow c \rightarrow f \rightarrow a |$$

Check each step is an edge in the graph:

1. $a - b$

2. $b - e$

3. $e - d$

4. $d - c$

5. $c - p$

6. $p - a$

Each of those edges appears in graph, and every vertex $\{a, b, c, d, e, f\}$ is visited exactly once before returning to a

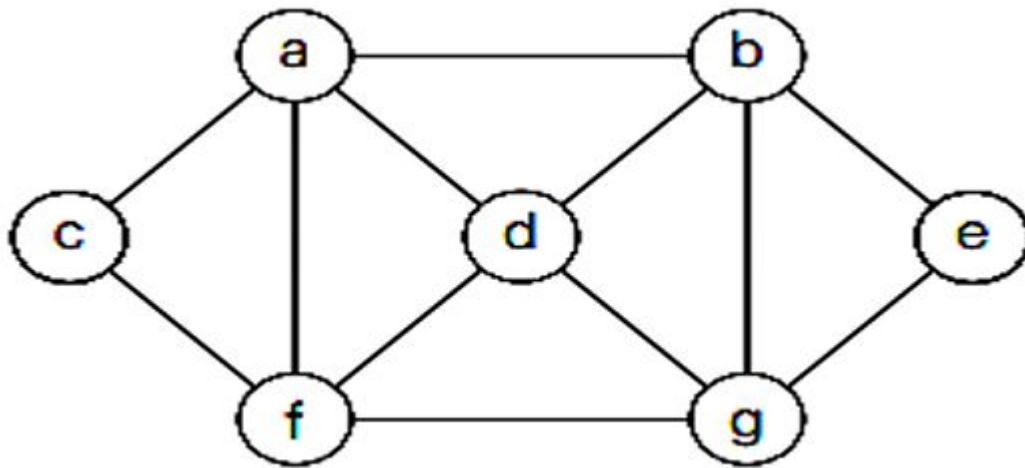
Hence this is a Hamiltonian cycle.

All vertices are visited exactly once (except the first/last vertex, a, which is revisited).

- **The last vertex (e) is connected to the first vertex (a).**
- **All edges exist in the graph**, meaning the cycle is valid.
- This is a valid **Hamiltonian cycle**

Apply backtracking to the given problem of finding a Hamiltonian circuit in the graph given in Fig.

- Backtracking is a recursive strategy used to solve problems incrementally. It explores all potential solutions and "**backtracks**" when a path doesn't lead to a valid solution.



- Goal is to find
- To find a **Hamiltonian circuit**, which:
- Starts and ends at the same vertex.

- $V = \{a, b, c, d, e, f, g\}$
- $a: [b, c, d, f]$
- $b: [a, d, e, g]$
- $c: [a, d, f]$
- $d: [a, b, c, f, g]$
- $e: [b, g]$
- $f: [a, c, d, g]$
- $g: [b, d, e, f]$

Steps:

- Start at node a
- Go to b
- Go to e
- Go to g
- Go to f
- Go to c
- Go to d
- Return to a

From backtracking, one valid circuit is:

$a \rightarrow b \rightarrow e \rightarrow g \rightarrow f \rightarrow c \rightarrow d \rightarrow a$

edges exist between all consecutive vertices:

a-b, b-e, e-g, g-f, f-c, c-d, d-a ✓

All vertices are included once.

The path returns to the start: ends at a ✓

Final Answer

Hamiltonian Circuit (using Backtracking):

$a \rightarrow b \rightarrow e \rightarrow g \rightarrow f \rightarrow c \rightarrow d \rightarrow a$

Difference Between NP-Hard and NP-complete

Aspect	NP-complete	NP-hard
Membership	In NP (decision problems verifiable in polynomial time)	Not required to be in NP (may be harder or not even decidable)
Relative Hardness	As hard as the hardest problems in NP	At least as hard as the hardest problems in NP
Poly-time Solver Implication	If any NP-complete problem is in P, then $P = NP$	If solved in polynomial time, yields a poly-time solver for all NP problems (but may go beyond NP)
Typical Examples	SAT, 3-SAT, Hamiltonian Cycle (decision), Clique (decision)	TSP (optimization), Halting Problem, general integer programming
Reduction Direction	Every NP problem reduces to any NP-complete problem (poly-time)	Every NP problem reduces to an NP-hard problem (poly-time)

